

Proposta de Arquitetura Técnica de Referência

Preparado para: Jabil Elaborado por: Victor Lucas — Engenheiro Fullstack Sênior Data: 03/07/2026

Sumário executivo

Este documento propõe o padrão de arquitetura e a stack tecnológica para os próximos projetos web fullstack, cobrindo backend (API) e frontend (painel/dashboard). O objetivo é registrar, antes de começarmos a codar, **como** vamos construir e **por que** — para que tanto o time técnico da Jabil quanto a Product Owner tenham visibilidade das decisões e possam questionar trade-offs antes que virem código.

Resumo em uma frase: **separamos claramente "regra de negócio" de "detalhe técnico"** (banco, cache, framework), o que nos dá testes rápidos e confiáveis, facilidade para trocar peças da infraestrutura sem reescrever o sistema, e um código mais previsível para qualquer engenheiro que entrar no time depois.

Todas as versões de dependências citadas abaixo foram verificadas como as mais recentes estáveis disponíveis em julho de 2026 — este não é um documento genérico reciclado, as escolhas refletem o estado atual do ecossistema.

1. Visão geral da arquitetura

Usamos **Clean Architecture + DDD tático** no backend e uma SPA React orientada a **estado de servidor** no frontend. A regra que guia todas as decisões abaixo:

As dependências apontam sempre para dentro. O núcleo de regras de negócio não sabe que Postgres, Redis ou NestJS existem — ele só conhece interfaces (contratos).

```
infra → domain → core
(detalhe) (regra) (genérico)
```

Por que isso importa para o negócio, não só para o time técnico:

- **Testes não dependem de banco/infra** → suíte de testes roda em segundos, não minutos, então rodamos em todo PR sem custo de CI alto.
- **Trocar Postgres por outro banco, ou Redis por outro cache, não obriga a reescrever regra de negócio** — só a peça de infraestrutura.

- **Regra de negócio fica em um único lugar** (a entidade/use case), não espalhada em controllers — menos duplicação de validação, menos bug de "esqueci de validar aqui também".

2. Backend

2.1 Stack

Área	Tecnologia	Versão (jul/2026)	Por que
Runtime	Node.js	24 LTS	LTS ativa em 2026; evitamos a v26 (ainda "Current", só vira LTS em out/2026) por estabilidade em produção
Framework	NestJS	11.1.x	Módulos, DI e decorators nativos já resolvem 80% do "encanamento" (validação, guards, interceptors), sem reinventar em cima do Express puro
Linguagem	TypeScript	6.0 (stable)	A 7.0 está em RC (compilador reescrito, ~10x mais rápido), mas ainda não recomendamos para produção — migramos assim que sair estável
ORM / Banco	Prisma	7.3 + PostgreSQL 18	Prisma 7 removeu a dependência do engine em Rust (motor mais simples de fazer deploy, menor cold start); Postgres 18 é a versão estável mais recente
Cache	Redis + ioredis	Redis 7.x / ioredis (latest)	Cache-aside para leituras pesadas (ex.: detalhes de um recurso com relações), TTL curto, sem acoplar a lógica de cache ao use case
Autenticação	JWT (RS256)	@nestjs/jwt + verificação via jose	Mantém o padrão RS256 (chave assimétrica, backend nunca expõe segredo de assinatura para quem só precisa verificar); trocamos passport-jwt por jose — menos dependências de runtime, API mais simples para um caso de uso único (verificar Bearer token)
Validação	Zod v4.4	—	Mesmo schema valida entrada HTTP e variáveis de ambiente; erros de validação viram resposta 400 estruturada, não exceção genérica

Área	Tecnologia	Versão (jul/2026)	Por que
Upload/Storage	@aws-s3 (latest v3)	Cloudflare R2	S3-compatível, custo de egress previsível, sem lock-in de storage proprietário
Testes	Vitest 4.1	—	Mesma engine (esbuild/Vite) do frontend — um único mental model de test runner no time inteiro
Package manager	pnpm 9	—	Instalação até 3-4x mais rápida, node_modules estrito (evita "dependência fantasma" — importar um pacote que não está no seu package.json só porque uma outra lib o trouxe)

2.2 Either — erro de negócio como valor, não como exceção

Use cases não usam `throw` para erro de regra de negócio. Retornam um tipo `Either<Erro, Sucesso>` :

```
export class Left<L, R> {
  constructor(readonly value: L) {}
  isLeft(): this is Left<L, R> { return true }
  isRight(): this is Right<L, R> { return false }
}

export class Right<L, R> {
  constructor(readonly value: R) {}
  isLeft(): this is Left<L, R> { return false }
  isRight(): this is Right<L, R> { return true }
}

export type Either<L, R> = Left<L, R> | Right<L, R>
```

Por quê: o próprio tipo de retorno do use case já documenta todo erro possível daquela operação. O TypeScript obriga quem chama a checar `isLeft()` antes de usar o valor — não existe caminho de erro "esquecido" silenciosamente, como acontece com `try/catch` genérico.

2.3 Use cases e repositórios como interface

```
@Injectable()
export class ApproveOrderUseCase {
  constructor(private ordersRepository: OrdersRepository) {} // interface, não imple

  async execute({ orderId }: ApproveOrderRequest): Promise<Either<ResourceNotFoundEr
    const order = await this.ordersRepository.findById(orderId)

    if (!order) {
```

```
    return left(new ResourceNotFoundError())
  }

  order.approve()
  await this.ordersRepository.save(order)

  return right({ order })
}
```

`OrdersRepository` é uma classe abstrata definida no `domain`. Quem implementa de verdade — com Prisma, com Redis, ou um fake em memória para teste — mora em `infra`. O use case nunca importa Prisma.

Ganho concreto: os testes de regra de negócio (a maioria dos testes do backend) rodam com um repositório fake em memória, sem subir Postgres nem Redis. Isso é o que permite `pnpm test` levar segundos em vez de minutos, mesmo com centenas de casos de teste.

2.4 Eventos de domínio

Ações que disparam efeitos colaterais entre módulos (ex.: aprovar um pedido dispara uma notificação) usam um pub/sub interno, não uma fila de mensageria externa:

```
protected addDomainEvent(event: DomainEvent): void {
  this._domainEvents.push(event)
  DomainEvents.markAggregateForDispatch(this)
}
```

O evento só é efetivamente disparado **depois** que o repositório confirma a escrita no banco — não corremos o risco de notificar algo que não foi de fato salvo.

Trade-off consciente: isso desacopla módulos (o módulo de pedidos não importa nada do módulo de notificações) sem a complexidade operacional de uma fila real (RabbitMQ/SQS). O custo é não termos garantia de entrega/retry automático — aceitável para o volume e criticidade inicial do projeto; se o volume crescer ou a entrega precisar ser garantida (ex.: notificação fiscal), migramos esse ponto específico para uma fila gerenciada, sem tocar no restante da arquitetura.

3. Frontend

3.1 Stack

Área	Tecnologia	Versão (jul/2026)	Por que
Framework	React 19.2	—	Server Components e Actions já maduros; menor boilerplate em formulários com <code>useActionState</code> quando fizer sentido
Build tool	Vite 8.1 (motor Rolldown)	—	Rolldown (Rust) substituiu o Rollup interno — builds e HMR sensivelmente mais rápidos que a geração anterior, sem mudar a forma de configurar o projeto
Linguagem	TypeScript 6.0	—	Mesma versão do backend — reduz atrito de "qual TS o projeto usa"
Roteamento	React Router v7	—	v6 entra em fim de suporte com o lançamento do v7/v8; v7 já traz tipagem automática de rotas e loaders, sem breaking change relevante vindo do v6
Estado de servidor	TanStack Query v5.101	—	Cache, refetch, invalidação e updates otimistas prontos — evita reinventar essa camada com <code>useEffect</code> manual
Formulários	<code>react-hook-form</code> + <code>@hookform/resolvers</code> + Zod v4	—	Mesmo schema de validação pode, em vários casos, ser compartilhado ou espelhado com o schema do backend
Design system	shadcn/ui sobre Base UI	—	Ver seção 3.2
Estilo	Tailwind CSS v4.3	—	Configuração 100% em CSS (<code>@theme</code> no lugar de <code>tailwind.config.js</code>), builds até 5x mais rápidos que a geração v3
Ícones	<code>lucide-react</code>	—	Mantido — biblioteca madura, tree-shakeable, sem mudança relevante de alternativa
Testes unitários/componente	Vitest 4.1 + Testing Library	—	Mesma engine do backend
Testes e2e	Playwright	—	Mantido — segue sendo o padrão de mercado para e2e em SPA

Área	Tecnologia	Versão (jul/2026)	Por que
Lint	ESLint 9 (flat config)	—	Configuração <code>eslint.config.js</code> única, sem <code>.eslintrc</code> legado

3.2 Design system: por que Base UI em vez de Radix em 2026

Nos nossos projetos de referência anteriores usamos shadcn/ui construído sobre **Radix UI**. Em 2026 o próprio shadcn/ui passou a suportar **Base UI** (mantido pelo time do MUI) como motor alternativo, com a mesma API de componentes.

Decisão: adotar Base UI para este projeto novo.

Critério	Radix UI	Base UI
Manutenção	Time pequeno, ritmo mais lento de releases em 2026	Time full-time do MUI, releases frequentes
Bundle	Referência histórica	Menor, segundo comparativos recentes
Migração	—	shadcn permite iniciar já com Base UI; componentes existentes em Radix continuam funcionando (não é obrigatório migrar projeto legado)

Continuamos com o padrão shadcn (copiar o componente para o repo, não importar de um pacote fechado) — isso não muda: o time mantém controle total de estilo e comportamento, sem depender de release de terceiro para customizar um componente.

```
/* src/styles/theme.css – Tailwind v4, config em CSS, sem tailwind.config.js */
@import "tailwindcss";

@theme {
  --color-primary: oklch(0.55 0.2 25);
  --color-primary-foreground: oklch(0.98 0 0);
  --radius: 0.5rem;
}
```

3.3 Estado de servidor e updates otimistas

Todo dado que vem da API passa por `useQuery` / `useMutation` do TanStack Query — não guardamos resposta de API em `useState` global nem em Context solto.

```
const { mutateAsync: approveOrder } = useMutation({
  mutationFn: approveOrderRequest,
  onMutate: async ({ orderId }) => {
    const previous = queryClient.getQueryData(['orders'])
    queryClient.setQueryData(['orders'], (old) => updateStatus(old, orderId, 'proces
```

```
    return { previous } // guardado para reverter se a API falhar
  },
  onError: (_err, _vars, context) => {
    queryClient.setQueryData(['orders'], context?.previous)
  },
})
```

Por que importa para a experiência do usuário: a interface responde imediatamente ao clique (não espera round-trip de rede para atualizar a tela), e reverte sozinha se o backend recusar a operação — sem o usuário perceber "tela travada" esperando resposta.

3.4 Formulários

```
const schema = z.object({
  email: z.string().email(),
})

const { register, handleSubmit, formState: { isSubmitting } } = useForm({
  resolver: zodResolver(schema),
})
```

Validação client-side e submissão desabilitada durante o envio (`isSubmitting`) evitam duplo clique e dado inválido chegando na API — a mesma validação de schema pode ser reaproveitada (ou espelhada) no DTO de entrada do backend, reduzindo a chance de regra de validação divergente entre as duas pontas.

4. Ferramentas transversais

Item	Escolha	Motivo
Package manager	pnpm 9 no backend e no frontend	Instala mais rápido, ocupa menos disco, e impede "dependência fantasma" (usar um pacote que não está declarado no seu <code>package.json</code>)
Formatação	Prettier	Mantido — zero debate de estilo de código em code review
CI	GitHub Actions (a configurar)	Rodar lint + testes + build em todo PR antes de merge — hoje o repo de referência não tinha CI configurado; para um projeto Jabil recomendamos já nascer com esse gate
Observabilidade	A definir com o time de infra da Jabil	Logs estruturados (JSON) já vêm de fábrica no <code>ConsoleLogger</code> do Nest 11; falta decidir para onde exportar (Datadog, CloudWatch, etc.)

5. O que fica em aberto / próximos passos

1. **Definir o domínio do primeiro módulo** — este documento cobre "como vamos construir", o próximo passo é modelar as entidades e casos de uso do problema de negócio específico.
 2. **CI/CD**: decidir pipeline e ambiente de deploy (não coberto aqui de propósito, depende de infraestrutura já existente na Jabil).
 3. **Observabilidade**: decidir ferramenta de logs/métricas em produção.
 4. **Confirmar com a Jabil** se há restrição de stack já homologada (ex.: banco de dados obrigatório, provedor de nuvem) que precise ajustar as escolhas acima.
-

Por que essa arquitetura, em uma frase para cada camada

- **Backend em Clean Architecture/DDD**: testamos regra de negócio sem infraestrutura, e trocamos peça de infraestrutura sem reescrever regra de negócio.
- **Frontend em React + TanStack Query**: a tela sempre reflete o estado real do servidor, com cache e updates otimistas resolvidos por uma lib madura em vez de código artesanal.
- **Design system shadcn (Base UI) + Tailwind v4**: componentes acessíveis por padrão, 100% customizáveis, sem depender de release de terceiro para mudar comportamento.
- **pnpm + TypeScript + Vitest no full stack**: um único conjunto de ferramentas e mental model entre backend e frontend, reduzindo a curva de troca de contexto de quem trabalha nos dois lados.